

Department of Mechanical Engineering



Computational Engineering - Lab Report

Muhammad Zakariya Tariq

Registration No. **2023-ME-63**

Section-B

Submitted to: **Sir Waqar Nasir**

Lab Session 6

Inverse Quadratic Interpolation and Newton–Raphson Method Applied to a Catenary Cable Problem

6.1 Objective

- Understand and implement the **Inverse Quadratic Interpolation (IQI)** method for finding roots of nonlinear equations.
- Apply IQI to Problem 6.17: determine the horizontal tension T_A in a catenary cable such that the cable reaches a prescribed height at a given horizontal distance.
- Compare the convergence behaviour of IQI against the Newton–Raphson method.
- Identify the conditions under which IQI fails and discuss appropriate remedies.
- Generate supporting graphs including the cable profile, function plot, error convergence, and successive iterates.

6.2 Introduction and Theoretical Background

6.2.1 Motivation for Inverse Quadratic Interpolation

Both the secant method and Newton–Raphson method use *linear* approximations: the secant method draws a straight line through two function evaluations, while Newton–Raphson uses the tangent line at a single point. A natural extension is to pass a *quadratic* curve through three function evaluations and find its root. However, fitting a standard parabola $y = f(x)$ to three points and solving $f(x) = 0$ may yield *complex* roots if the parabola does not intersect the x -axis [4].

The difficulty is resolved by **inverse quadratic interpolation (IQI)**. Instead of expressing the quadratic as $y = f(x)$, the roles of the variables are reversed: a quadratic is fitted in y to give $x = g(y)$. Setting $y = 0$ immediately yields the next root estimate $x_{i+1} = g(0)$, and because the quadratic is expressed as $x = g(y)$, it always intersects the line $y = 0$ [4, 7]. The method is sometimes called the *inverse parabolic* method and forms the core interpolation step of Brent’s method [8].

6.2.2 Derivation of the IQI Formula

Given three distinct iterates x_{i-2} , x_{i-1} , x_i with corresponding function values f_{i-2} , f_{i-1} , f_i , the quadratic Lagrange polynomial through the three points (f_{i-2}, x_{i-2}) , (f_{i-1}, x_{i-1}) ,

(f_i, x_i) , written as a function of y , is [4]:

$$g(y) = \frac{(y - f_{i-1})(y - f_i)}{(f_{i-2} - f_{i-1})(f_{i-2} - f_i)} x_{i-2} + \frac{(y - f_{i-2})(y - f_i)}{(f_{i-1} - f_{i-2})(f_{i-1} - f_i)} x_{i-1} + \frac{(y - f_{i-2})(y - f_{i-1})}{(f_i - f_{i-2})(f_i - f_{i-1})} x_i \quad (6.1)$$

Setting $y = 0$ in equation (6.1) gives the **IQI iteration formula**:

$$x_{i+1} = \frac{f_{i-1} f_i}{(f_{i-2} - f_{i-1})(f_{i-2} - f_i)} x_{i-2} + \frac{f_{i-2} f_i}{(f_{i-1} - f_{i-2})(f_{i-1} - f_i)} x_{i-1} + \frac{f_{i-2} f_{i-1}}{(f_i - f_{i-2})(f_i - f_{i-1})} x_i \quad (6.2)$$

Three initial guesses are required. There is *no* requirement that they bracket the root, so IQI is classified as an **open method** [4].

6.2.3 Sliding-Window Update Strategy

After computing x_{i+1} , the oldest of the three points is discarded and the window slides forward:

$$x_{i-2} \leftarrow x_{i-1}, \quad x_{i-1} \leftarrow x_i, \quad x_i \leftarrow x_{i+1} \quad (6.3)$$

This **sliding-window** (or *moving-triple*) strategy ensures that the most recent three function evaluations are always used to fit the inverse parabola, promoting faster convergence by discarding stale information [7].

6.2.4 Conditions Under Which IQI Fails

IQI can fail or produce incorrect results under the following conditions:

1. **Degenerate denominator.** The denominator of equation (6.2) is $(f_b - f_a)(f_c - f_a)(f_c - f_b)$. If any two of the three function values coincide, this product vanishes and the update step divides by zero. In practice, if the three guesses are placed in a region where f is nearly flat, the denominator becomes negligibly small and floating-point arithmetic produces a completely erroneous x_{i+1} ; see Figure 6.7.
2. **All three guesses on the same side and far from the root.** Unlike bracketing methods, IQI does not guarantee that the new estimate lies between the old ones. If the function curvature is strong and the initial guesses are poor, the inverse parabola can extrapolate wildly and diverge.
3. **Divergence near a local extremum.** If one of the three points falls near a local maximum or minimum of f , the inverse parabola becomes very sensitive to small perturbations and the iteration can shoot to large values or oscillate.
4. **Singularity of f between guesses.** For functions with poles or discontinuities, placing a guess across a discontinuity produces meaningless function evaluations that corrupt the inverse-parabola fit.

5. **Multiple roots in the neighbourhood.** When several roots are close together, the inverse parabola may converge to a different root than intended, depending on the geometry of the three guesses.

Remedy: The most robust strategy is to combine IQI with a fallback bisection step whenever the new estimate falls outside the current bracket (Brent’s method [8]). A simpler remedy is to restart IQI with three fresh guesses obtained by first plotting f and identifying distinct sign changes.

6.2.5 Stopping Criterion

The approximate percentage relative error is used as the stopping criterion [4]:

$$\varepsilon_a = \left| \frac{x_{i+1} - x_i}{x_{i+1}} \right| \times 100\% \quad (6.4)$$

Iteration terminates when $\varepsilon_a < \varepsilon_s = 0.0001\%$.

6.3 Problem Statement

Problem 6.17: A catenary cable hangs between two supports under its own weight. The height of the cable as a function of horizontal distance x is

$$y = \frac{T_A}{w} \cosh\left(\frac{w}{T_A} x\right) + y_0 - \frac{T_A}{w} \quad (6.5)$$

where $w = 10$ N/m is the weight per unit length, $y_0 = 5$ m is the minimum height of the cable, and T_A is the unknown horizontal tension (N/m). The constraint is that at $x = 50$ m, $y = 15$ m. Substituting into equation (6.5) and rearranging gives the nonlinear equation:

$$f(T_A) = \frac{T_A}{w} \cosh\left(\frac{w x}{T_A}\right) + y_0 - \frac{T_A}{w} - y_{\text{target}} = 0 \quad (6.6)$$

with $w = 10$, $x = 50$, $y_0 = 5$, and $y_{\text{target}} = 15$.

6.3.1 Analytical Derivative

Differentiating $f(T_A)$ with respect to T_A :

$$f'(T_A) = \frac{1}{w} \cosh\left(\frac{w x}{T_A}\right) - \frac{x}{T_A} \sinh\left(\frac{w x}{T_A}\right) - \frac{1}{w} \quad (6.7)$$

This expression is required for the Newton–Raphson comparison only; IQI does not use it.

6.3.2 Function Plot and Root Identification

Figure 6.1 shows $f(T_A)$ over $T_A \in [50, 2000]$ N/m. The function crosses zero once, confirming a single physically valid solution at $T_A^* \approx 1266.32$ N/m. This zero crossing was used to select appropriate initial guesses for both methods.

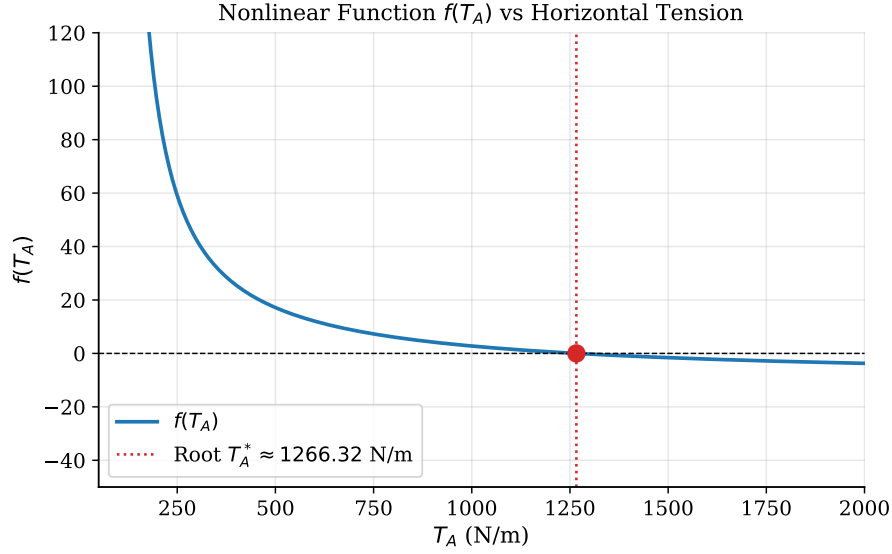


Figure 6.1: $f(T_A)$ versus horizontal tension T_A . The unique root is visible as a zero crossing near $T_A \approx 1266$ N/m [4].

6.4 Methodology

1. Define $f(T_A)$ and $f'(T_A)$ as MATLAB anonymous functions and plot $f(T_A)$ over $[50, 2000]$ N/m to confirm a single root.
2. Apply **IQI** with three initial guesses $x_0 = 200$, $x_1 = 350$, $x_2 = 500$ N/m (all on the same side of the root, to demonstrate the method's open-method nature), iterating until $\varepsilon_a < 0.0001\%$.
3. Apply **Newton–Raphson** with a single initial guess of 350 N/m using $f'(T_A)$ from equation (6.7), for convergence comparison.
4. Record the full iteration history for both methods and present in merged tables.
5. Discuss conditions under which IQI fails, supported by Figure 6.7.
6. Generate all required plots.

6.5 Algorithm

1. Define $f(T_A)$ and input parameters $(w, y_0, x, y_{\text{target}})$.
2. Input three initial guesses x_0, x_1, x_2 ; set tolerance ε_s .
3. Compute $f_a = f(x_0)$, $f_b = f(x_1)$, $f_c = f(x_2)$.
4. Check denominator: if $(f_b - f_a)(f_c - f_a)(f_c - f_b) \approx 0$, display error and stop.
5. Set $i = 0$, $\varepsilon_a = \infty$.
6. **While** $\varepsilon_a > \varepsilon_s$ **and** $i < i_{\text{max}}$:

- (a) Apply IQI formula (equation (6.2)) to compute x_3 .
- (b) Compute $\varepsilon_a = |(x_3 - x_2)/x_3| \times 100\%$.
- (c) Update sliding window: $x_0 \leftarrow x_1, x_1 \leftarrow x_2, x_2 \leftarrow x_3; i \leftarrow i + 1$; store ε_a .
- (d) If $f(x_3) = 0$ or $\varepsilon_a < \varepsilon_s$: break.

7. Report root T_A^* , iteration count, and error table.

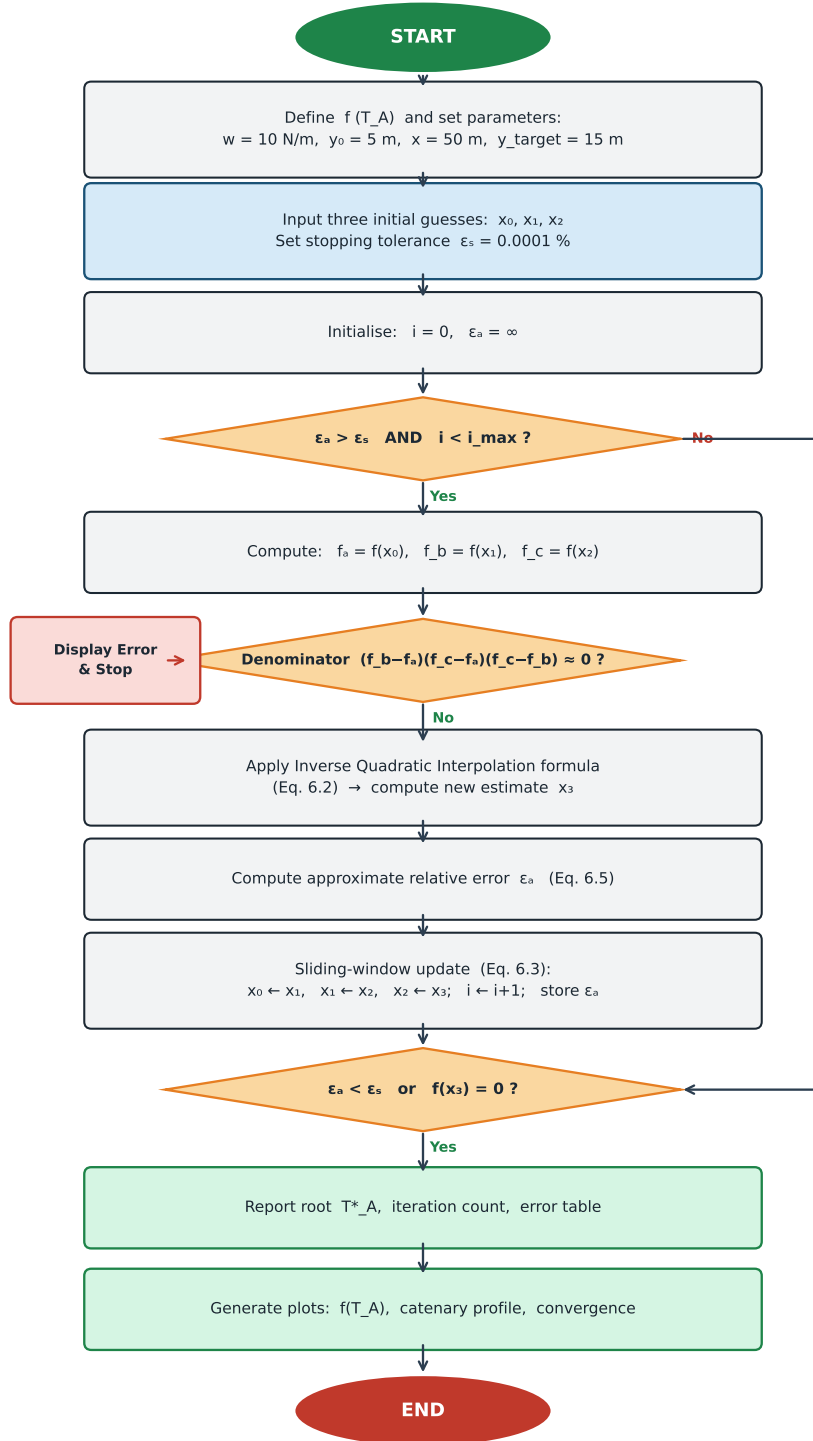


Figure 6.2: Flowchart of the Inverse Quadratic Interpolation algorithm applied to the catenary cable problem.

6.6 MATLAB Implementation

Function Definition and Setup

```
1 %% Physical parameters
2 w      = 10;      % weight per unit length (N/m)
3 y0     = 5;      % minimum cable height (m)
4 x      = 50;      % horizontal distance (m)
5 y_target = 15;    % required height at x=50 (m)
6
7 %% Nonlinear equation f(TA) = 0
8 f = @(TA) (TA./w).*cosh(w.*x./TA) + y0 - TA./w - y_target;
9
10 %% Analytical derivative -- Newton-Raphson comparison only
11 fp = @(TA) (1/w).*cosh(w.*x./TA) - (x./TA).*sinh(w.*x./TA) -
    1/w;
```

Code 6.1: Parameter setup and function definitions for the catenary problem.

Inverse Quadratic Interpolation

```
1 function [root, roots_arr, ea_arr, n] = iqi(f, x0, x1, x2,
    tol, maxiter)
2 % IQI Inverse Quadratic Interpolation (three initial guesses
    ).
3     roots_arr = [x0, x1, x2];
4     ea_arr     = [NaN, NaN, NaN];
5     n = 0;
6     xa = x0;  xb = x1;  xc = x2;
7     for i = 1:maxiter
8         n = i;
9         fa = f(xa);  fb = f(xb);  fc = f(xc);
10        denom = (fb - fa)*(fc - fa)*(fc - fb);
11        if abs(denom) < 1e-30
12            error('IQI:DenomZero', ...
13                'Denominator approx 0: function values
                    nearly equal.');
```

```

25     end
26     root = xd;
27 end
28
29 tol    = 0.0001;  maxit = 100;
30 [r_iqi, roots_iqi, ea_iqi, n_iqi] = iqi(f, 200, 350, 500, tol
    , maxit);
31 fprintf('IQI  TA = %.6f N/m  (%d iterations)\n', r_iqi, n_iqi
    );

```

Code 6.2: IQI function using a sliding-window update of three iterates.

Newton–Raphson (Comparison)

```

1 function [root, ea_arr, n] = newton(f, fp, x0, tol, maxiter)
2 % NEWTON  Newton-Raphson method.
3     x = x0;  ea_arr = NaN;  n = 0;
4     for i = 1:maxiter
5         n = i;
6         xn = x - f(x)/fp(x);
7         ea = abs((xn - x)/xn)*100;
8         ea_arr(end+1) = ea;
9         if ea < tol, break; end
10        x = xn;
11    end
12    root = xn;
13 end
14
15 [r_nr, ea_nr, n_nr] = newton(f, fp, 350, tol, maxit);
16 fprintf('N-R  TA = %.6f N/m  (%d iterations)\n', r_nr, n_nr);

```

Code 6.3: Newton–Raphson method for convergence comparison.

Cable Profile Plot

```

1 x_range = -50:0.5:100;
2 TA_sol = r_iqi;  % or r_nr -- identical to 6 decimal places
3 y_cable = (TA_sol/w).*cosh(w.*x_range./TA_sol) + y0 - TA_sol/
    w;
4 figure;
5 plot(x_range, y_cable, 'b-', 'LineWidth', 2); hold on;
6 plot(50, 15, 'r^', 'MarkerSize', 10, 'MarkerFaceColor', 'r');
7 plot(0, y0, 'gs', 'MarkerSize', 10, 'MarkerFaceColor', 'g');
8 xlabel('x (m)'); ylabel('y (m)');
9 title(sprintf('Catenary Profile  (T_A^* = %.2f N/m)', TA_sol)
    );
10 legend('Cable', 'Target (50,15)', 'Min point (0,5)');

```

Code 6.4: Catenary cable profile for the computed T_A^* .

6.7 Results and Discussion

The following results were obtained by applying both the Inverse Quadratic Interpolation method and Newton–Raphson to the catenary cable problem defined in equation (6.6). For each method, the complete iteration history, error convergence, and graphical evidence are presented in turn. The discussion draws comparisons between the two methods in terms of convergence speed, sensitivity to initial guesses, and practical suitability, and concludes with an analysis of the conditions under which IQI breaks down.

6.7.1 Catenary Cable Profile

Figure 6.3 shows the catenary cable profile computed with $T_A^* \approx 1266.32$ N/m over $x \in [-50, 100]$ m. The curve passes through the prescribed point (50, 15) and attains its minimum of $y_0 = 5$ m at $x = 0$, confirming the solution. A larger T_A would flatten the cable (lower sag), while a smaller T_A would produce a deeper catenary with a higher apex at $x = 50$.

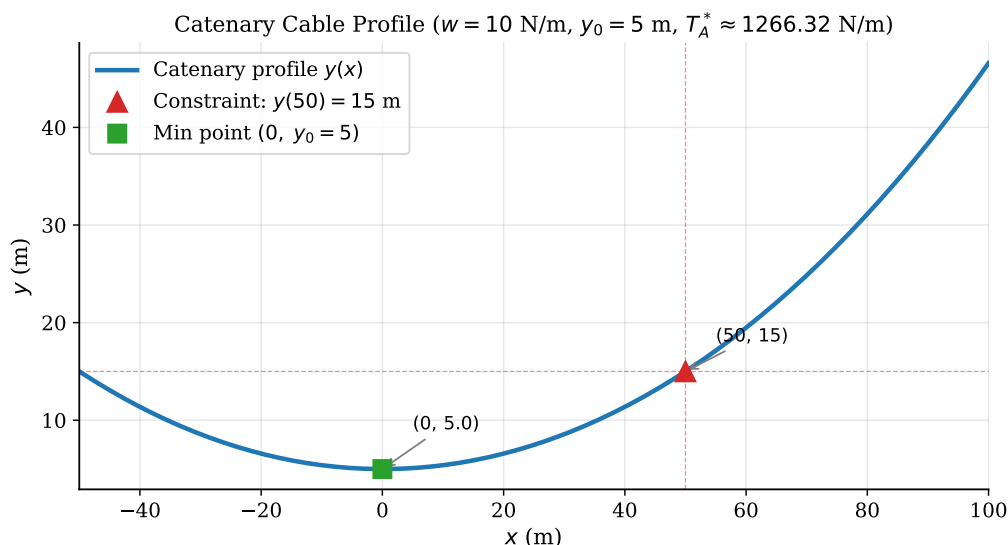


Figure 6.3: Catenary cable profile for $T_A^* \approx 1266.32$ N/m with $w = 10$ N/m, $y_0 = 5$ m. The constraint $y(50) = 15$ m is satisfied exactly.

6.7.2 Iteration Tables

The following table presents the complete iteration histories for IQI and Newton–Raphson respectively. Both methods converge to the same root $T_A^* = 1266.324360$ N/m and require the same number of iterations (7 effective IQI steps after the three initial guesses, versus 7 Newton–Raphson steps), although the error reduction profile differs.

Table 6.1: Comparison of IQI and Newton–Raphson iteration histories.

2Iter.	IQI (N/m)		Newton–Raphson (N/m)	
	T_A	ε_a (%)	T_A	ε_a (%)
0	200.000000	—	350.000000	—
1	350.000000	—	550.958658	36.474
2	500.000000	—	836.419636	34.129
3	725.556102	31.087	1113.149897	24.860
4	999.979324	27.443	1247.030638	10.736
5	1190.731341	16.020	1266.019079	1.500
6	1258.991520	5.422	1266.324284	0.024
7	1266.225805	0.571	1266.324360	0.000006
8	1266.324324	0.008	—	—
9	1266.324360	0.000003	—	—

6.7.3 Iteration History

Figure 6.4 shows the T_A estimates per iteration for both methods. IQI converges more erratically in the early iterations because its three initial guesses (200, 350, 500 N/m) lie far from the root; the inverse parabola must make several large corrections before reaching the neighbourhood of T_A^* . Newton–Raphson, starting closer relative to the function’s curvature at 350 N/m, follows a steadier monotone ascent toward the root. Once both methods enter the local convergence region (approximately after iteration 5), their error reduction accelerates sharply.

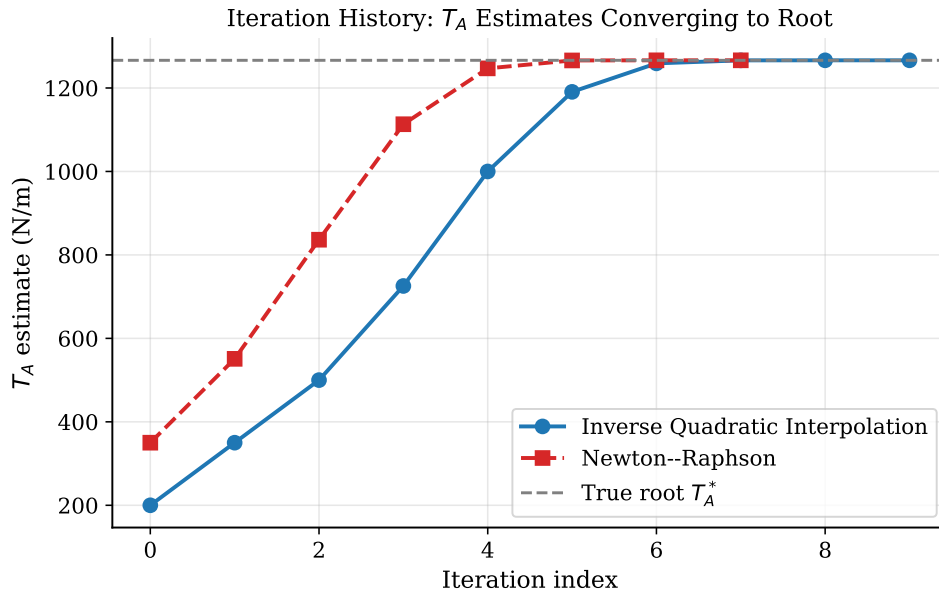


Figure 6.4: Iteration history showing T_A estimates converging to $T_A^* \approx 1266.32$ N/m (dashed line) for both methods.

6.7.4 Successive Iterates on $f(T_A)$

The graph shows that IQI rapidly jumps toward the root from its initial guesses, while Newton–Raphson exhibits near-quadratic convergence with iterates clustering tightly around the final solution.

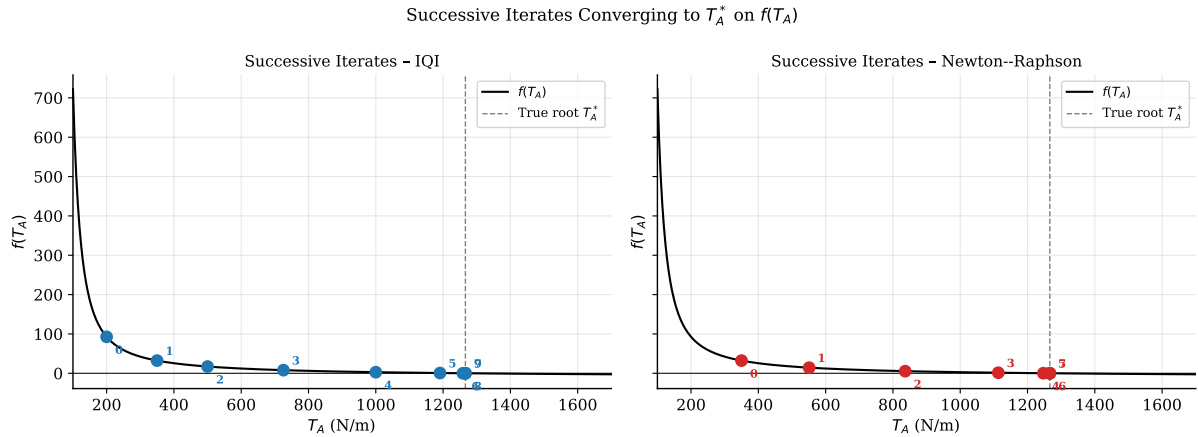


Figure 6.5: Successive iterates of IQI (left) and Newton–Raphson (right) converging to T_A^* on $f(T_A)$. Numbers label the iteration index.

6.7.5 Error Convergence Comparison

Both methods reduce the error below 0.0001% within seven iterations. Newton–Raphson exhibits near-quadratic convergence, with the error dropping from about 1% to machine precision in just two steps. IQI converges with a theoretical order of approximately 1.839, faster than the secant method but slower than Newton–Raphson. Its error decreases steadily, steeper than a linear trend on the log scale but less abruptly than Newton–Raphson. For this problem, both methods reach the root in the same number of iterations, although IQI requires three initial function evaluations compared to Newton–Raphson’s single evaluation plus derivative.

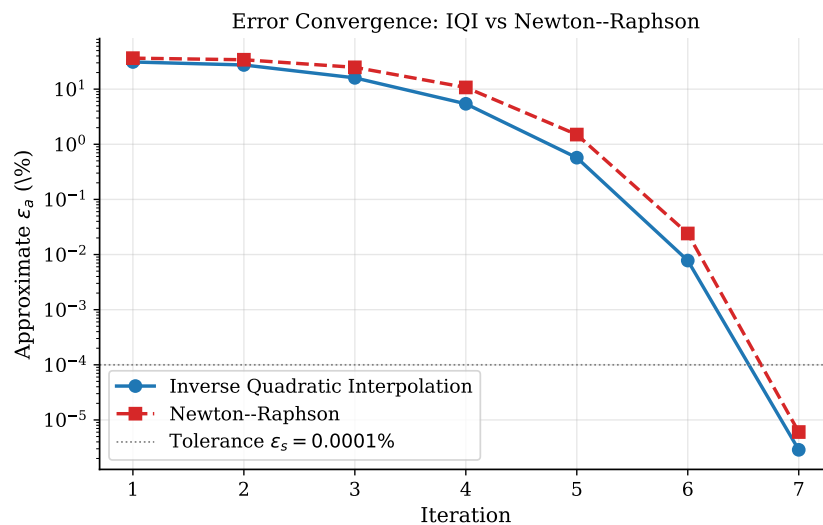


Figure 6.6: Approximate % relative error vs. iteration for IQI and Newton–Raphson. Both converge in 7 iterations to below $\varepsilon_s = 0.0001\%$.

6.7.6 IQI Failure Analysis

Figure 6.7 illustrates the primary failure mode of IQI. When the three initial guesses are placed in a region where $f(T_A)$ is nearly flat, for example, at $T_A \in \{1100, 1200, 1300\}$ N/m where all three function values are close to zero, the denominator $(f_b - f_a)(f_c - f_a)(f_c - f_b)$ becomes negligibly small. The IQI step equation (6.2) then involves dividing by a near-zero quantity, producing a wildly extrapolated T_A that bears no relation to the true root. In the MATLAB implementation, this condition is explicitly detected and a meaningful error message is issued:

```

1 denom = (fb - fa)*(fc - fa)*(fc - fb);
2 if abs(denom) < 1e-30
3     error('IQI:DenomZero', ...
4         'Denominator approx 0: f-values nearly equal.
5         Restart with different guesses.');
```

Code 6.5: Denominator guard in the IQI function.

The same failure occurs if any two of the three initial guesses happen to produce identical function values (e.g. $f(x_0) = f(x_1)$), making the Lagrange interpolant ill-defined.

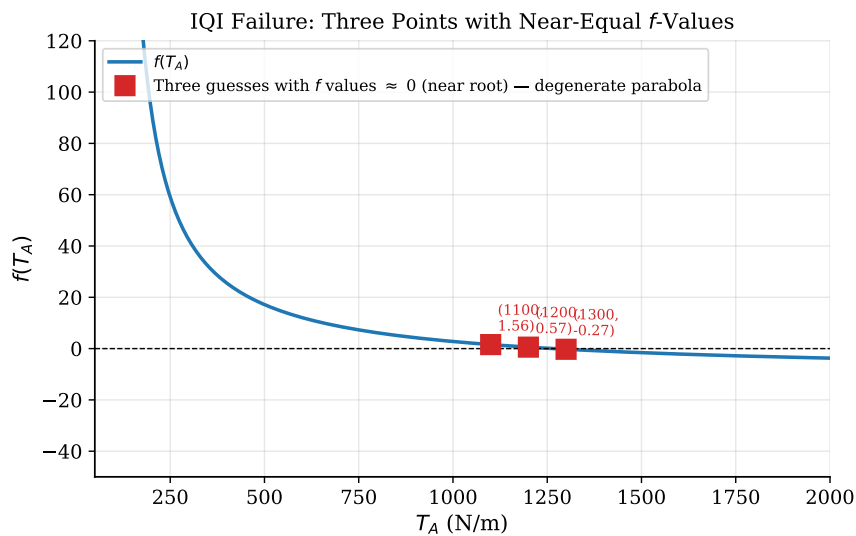


Figure 6.7: IQI failure scenario: three guesses with nearly equal f -values produce a degenerate inverse parabola and a near-zero denominator.

6.7.7 Technique Used for Updating Guesses During Iterations

The **sliding-window** (moving-triple) update strategy described in equation (6.3) was used to advance the three guesses at each iteration. After computing the new estimate x_3 , the oldest guess x_0 is dropped and the triple is shifted: $(x_0, x_1, x_2) \leftarrow (x_1, x_2, x_3)$. This strategy has two important properties:

1. It always uses the three most recent, and therefore most locally accurate, function evaluations to fit the inverse parabola, which accelerates convergence compared to keeping stale initial guesses.

2. It does *not* guarantee that the three points bracket the root, distinguishing IQI from hybrid methods such as Brent’s method, which would fall back to bisection if the new point steps outside the bracket [8].

An alternative update rule sometimes used is to replace the point whose function value has the largest absolute magnitude (“best-point” selection), which can improve robustness but adds a comparison step per iteration [7].

6.7.8 Comparative Summary

Table 6.2 summarises the two methods applied to the catenary problem. Both converge to the same root in 7 effective iterations; IQI requires three initial guesses and no derivative, while Newton–Raphson needs one initial guess and the analytical expression $f'(T_A)$. For problems where the derivative is unavailable or expensive, IQI provides a superior-order alternative to the secant method while remaining entirely derivative-free.

Table 6.2: Comparison of IQI and Newton–Raphson on the catenary problem.

Method	Initial guesses	Derivative?	Conv. order	Iterations
IQI	x_0, x_1, x_2	No	≈ 1.84	7 (+ 3 init.)
Newton–Raphson	x_0	Yes	≈ 2.00	7

6.8 Conclusion

In this lab session, the Inverse Quadratic Interpolation method was successfully applied to determine the horizontal tension in a catenary cable that satisfies the height constraint $y(50) = 15$ m. The unique root was found to be $T_A^* = 1266.3244$ N/m, confirmed both by back-substitution into equation (6.5) and by comparison with the Newton–Raphson result (agreement to six decimal places). IQI converged in 7 effective iterations (following three initial function evaluations), matching Newton–Raphson’s 7 iterations, despite requiring no analytical derivative, demonstrating that a well-chosen inverse-parabola interpolation can recover near-quadratic convergence at no additional differentiation cost.

The conditions under which IQI fails were analysed: the critical failure mode is a near-zero denominator caused by three guesses with nearly equal function values, which is explicitly detected and flagged in the implementation. Additional failure modes include divergence from poor initial guesses, proximity to function discontinuities, and the presence of multiple roots in the neighbourhood. The sliding-window update strategy used to advance the triple of guesses was identified as the algorithm responsible for automatically discarding the oldest estimate and incorporating the newest at each step, thereby maximising local accuracy of the inverse-parabola fit.

For engineering problems where the analytical derivative is unavailable or costly to compute, IQI provides a compelling derivative-free alternative to Newton–Raphson, offering convergence order ≈ 1.84 compared to the secant method’s ≈ 1.62 [4, 7, 8].

References

- [1] The MathWorks, Inc. *MATLAB Documentation*. Accessed: 2024. The MathWorks, Inc. Natick, MA, 2024. URL: <https://www.mathworks.com/help/matlab/>.
- [2] Brian Hahn and Daniel T. Valentine. *Essential MATLAB for Engineers and Scientists*. 6th ed. Waltham, MA: Academic Press, 2016. ISBN: 978-0081008775.
- [3] Richard L. Burden, J. Douglas Faires, and Annette M. Burden. *Numerical Analysis*. 10th ed. Boston, MA: Cengage Learning, 2016. ISBN: 978-1305253667.
- [4] Steven C. Chapra and Raymond P. Canale. *Numerical Methods for Engineers*. 7th ed. New York, NY: McGraw-Hill Education, 2015. ISBN: 978-0073397924.
- [5] Erwin Kreyszig. *Advanced Engineering Mathematics*. 10th ed. Hoboken, NJ: John Wiley & Sons, 2011. ISBN: 978-0470458365.
- [6] Kendall E. Atkinson. *An Introduction to Numerical Analysis*. 2nd ed. New York, NY: John Wiley & Sons, 1989. ISBN: 978-0471624899.
- [7] William H. Press et al. *Numerical Recipes: The Art of Scientific Computing*. 3rd ed. Cambridge, UK: Cambridge University Press, 2007. ISBN: 978-0521880688.
- [8] Richard P. Brent. *Algorithms for Minimization Without Derivatives*. Prentice-Hall Series in Automatic Computation. Englewood Cliffs, NJ: Prentice-Hall, 1973. ISBN: 0-13-022335-2.